PROJECT-BASED LEARNING

PHASE-II EVALUATION

Rootrix Virtual File System

Team Rootrix — Team ID: DSCPP-III-2026-T123

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T123
Semester: 3rd
Section: G / H / ML2 / DS2
Domain: Data Structures and OOPs

Team Members

1. Tanish Naithani – 25100011477
2. Tushar Singh – 2510011704
3. Rounak Tayal – 2510370114
4. Aditya Rawal – 2510380204

Mentor

Mr. Akshay Rajput

Interactions completed: 4

Tanish (Team Lead): Directory Tree — Aditya: Blocks — Tushar: Metadata, Tests — Rounak: CLI

Phase-I Recap & Feedback Incorporated

Phase-I Status

Phase-I was the proposal: a C++ Virtual File System with a folder tree, block storage, a bitmap, hash search and security. Coding started in Phase-II.

Feedback Received

- Private folders need security
- Show created files on the PC
- Rahul / rahul differ from Windows

Changes Incorporated

- Added lock / unlock with a password
- Changes copied to `vfs_root`
- Names now ignore case, like NTFS

We also added saving to `vfs_save.txt`, so files stay after restart.

Problem Statement & Finalized Objectives

Problem Statement

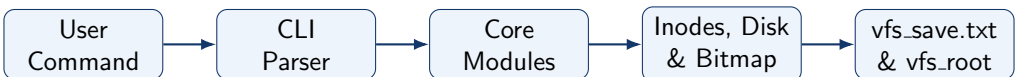
We use file systems daily but rarely see how they work inside. We build our own on a virtual disk, so inodes, blocks and folder trees can be traced step by step.

Finalized Objectives

1. Store data on a 64-block $\times$ 32-character disk with a bitmap
2. Represent every file and folder as an inode (table of 32)
3. Provide 19 commands, read-only files and locked folders
4. Find files by path with a hash index; keep data after restart

Phase-II: core system + 19 commands. Phase-III: LRU cache and journaling.

Updated System Architecture / Workflow



Major Components

- CLI parser: command, name, text
- Tree, Block Manager, Hash Index
- Inodes, disk, bitmap in RAM

Changes Since Phase-I

- Inodes with parent numbers
- Save file and real folder copy
- CLI rebuilt with OOP classes

Every command checks first, then changes; after it, the hash index is rebuilt and data is saved.

Module Design & Functional Requirements

Major Modules

1. Directory Tree (Tanish)
2. Block Manager (Aditya)
3. Metadata & Hash Index (Tushar)
4. OOP CLI & Real Folder (Rounak)

Functional Requirements

- Create, edit, copy, move, delete
- Refuse writes when disk is full
- Protect read-only / locked items
- Keep data after restart

Module Dependency

`real_folder` → `directory_tree` → `block_manager` → `metadata` → `cli` → `main`

Only the Block Manager touches the disk and the bitmap.

Technology Stack & Development Methodology

Technology Stack

- Programming: C++
- Database: None (own save file)
- Framework: None (std library)
- Development Tool: VS Code, g++
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Built step by step: 3 commands, then 10, then 19, testing after each step.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C++	Arrays (inode table, 2D disk), bitmap, tree via parent numbers, hashing, recursion	Tree, Blocks, Index
OOPs with C++	Classes in the CLI module: abstract class `Command`, multilevel inheritance, virtual `execute()` (polymorphism), `Shell` class (encapsulation, constructor/destructor)	CLI (Rounak)
Operating Systems	Inodes, block allocation, free-space bitmap, tree directories, protection	Tree, Blocks
DBMS	Indexing (hash index on paths), saving to a file; no SQL	Index, Blocks

Important: Mention only those concepts that have been **actually implemented** in the project.

Character i of a file goes to block $i/32$, position $i \bmod 32$.

Development & Implementation Progress

Module	Status	Work Completed
Directory Tree	Completed	Inodes, paths; mkdir, touch, cd, mv, chmod
Block Manager	Completed	Disk, bitmap, save/load; write, cat, append, cp, df
Metadata, Index & CLI	Completed	ls, rm, stat, find; OOP CLI; pwd, lock, unlock
LRU Cache & Journaling	Pending	Phase-III: block cache, crash and recover

Overall Project Completion: Phase-II 100% — Overall about *75%* (team estimate)

All 19 Phase-II commands work and are tested.

Working Prototype / Demonstration

Implemented Features

- 19 working commands
- Read-only and locked folders
- Hash index search (find)
- Data kept after restart

Prototype / Output

```
vfs:/> mkdir docs
Folder created:  docs (inode 1)
vfs:/> cd docs
vfs:/docs> touch a.txt
File created:  a.txt (inode 2)
vfs:/docs> write a.txt hello world
Wrote 11 bytes to a.txt
vfs:/docs> ls
[FILE]  a.txt  (inode 2, 11 bytes, blocks:  0)
```

Live demo: write, chmod ro, lock, find, df, then restart the program.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Disk full	Big write refused, old text kept	"disk is full", old text kept	Pass
Read-only / lock	Changes blocked; cd needs password	Refused; right password works	Pass
Restart	Files, locks, read-only return	All restored from save file	Pass
Rahul / rahul	Second name refused	"rahul already exists"	Pass

Results / Validation

Test program: 16 tests, 39 checks, all pass (real values from inodes and disk). Compiles with C++11 and C++17 with no warnings; AddressSanitizer found no errors.

Run on Linux (g++ 13); to be re-run on every team laptop.

GitHub Repository & Development Activity

Repository Information

- GitHub Link:
  github.com/adityarawal1985-bit/virtual-file-system
- README.md: In progress
- Contributors: all 4 members
- Repository Status: 25 commits
- Branches: main

Development Activity

- *Regular commits maintained*
- *Meaningful commit messages*
- Every member commits individually
- *Latest implementation uploaded*
- *Documentation maintained*

Each member commits his own source file from his own account.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Tanish Naithani	Lead, Dir. Tree	Inode table, paths; mkdir, touch, cd, mv, chmod	4
Aditya Rawal	Block Manager	Disk, bitmap, save/load; write, cat, append, cp, df	9
Tushar Singh	Metadata, Tests	Hash index; ls, rm, stat, find; 16 tests	6
Rounak Tayal	CLI & Docs	OOP CLI (Command classes, Shell), real folder; pwd, lock, unlock	6

Each member owns one file and about five commands.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Old g++: no `<filesystem>`
- Windows blocked our exe
- Data lost on restart

Solutions / Learning

- Used _mkdir, remove; save file
- Inheritance removed a 19-way if/else

Phase-III Roadmap

1. LRU block cache (Aditya)
2. Journaling (Tushar)
3. crash / recover (Rounak)
4. Growing disk (Tanish)
5. README, final tests (All)

The disk holds 2048 characters; large files are refused safely for now.

Expected Outcomes & References

Expected Final Outcomes

- VFS with cache and journaling
- Measurable automated tests
- Model of real file systems
- Disk size easy to increase

References

1. E. Balagurusamy, *Object Oriented Programming with C++*
2. Y. Kanetkar, *Data Structures Through C++*
3. GeeksforGeeks (C++ OOP, data structures)
4. cppreference.com (C++ library reference)
5. Microsoft Learn (NTFS overview)

Thank You
Questions & Suggestions

A tested file system built from scratch with inodes, blocks and hashing.